RAJALAKSHMI
ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

BONAFIDE CERTIFICATE

NAME ..... Ashleena Gladice Princely .....

ACADEMIC YEAR 2026-2027 SEMESTER ..... V ..... BRANCH ..... CSD .....

UNIVERSITY REGISTER No. 2116241701006

Certified that this is the bonafide record of work done by the above student in the

(IT23E31)
Graphics & Multimedia Laboratory during the year 2026 - 2027

30/10/26

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on .....

Internal Examiner

External Examiner

Scanned with OKEN Scanner

INDEX

NAME Ashleena Gladice Princely ~~SUBJECT~~ (241701006)

STD. _____ DIV. _____ ROLL NO. _____ ~~SCHOOL~~ CSD

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1.	3	Exp1 - Bresenham's Line Drawing Principle Bresenham's square Algorithm Rectangle using Bresenham's algorithm Olympic logo using mid-point algorithm Butterfly using mid-point algorithm Traffic signal using mid-point algorithm Smiley Face using mid point algorithm	17/7/25	
2.		Exp 2 Flood Fill Algorithm		
3.		Exp 3 Transformation of Triangle Transformation of Rectangle		
4.		Exp 4 Polygon clipping		
5.		Exp 5 Transformation of cube & Transformation of Pyramid.		
6.		Exp 6 Creating and Rendering 3D objects.		
7.		Exp 7. Creating a multimedia application that integrates images, sounds and video in a simple UI		
8.		Exp 8 Program that Captures Audio/Video from a webcam/microphone and displays it.		

Scanned with OKEN Scanner

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
9.		Exp9-To create an Application to send and Receive Multimedia Messages		
10.		Exp10-To create a 3D object in Blender and apply shading to it		

Completed

Scanned with OKEN Scanner

17/02/26 Experiment - 1 Bresenham's Line Drawing Principle

aim: to write and Execute a python program to Implement Bresenham's Line Drawing Algorithm for Drawing a straight line where the slope $|m| < 1$.

program:-

```
def bresenhams (x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    p = 2 * abs(dy) - abs(dx)
    x = x1
    y = y1
    y_step = 1 if dy >= 0 else -1
    print("Points on the line:")
    print(f"({x}, {y})")
    while x < x2:
        x += 1
        if p < 0:
            p = p + 2 * abs(dy)
        else:
            y += y_step
            p = p + 2 * abs(dy) - 2 * abs(dx)
        print(f"({x}, {y})")
x1 = int(input("Enter x1:"))
x2 = int(input("Enter x2:"))
y1 = int(input("Enter y1:"))
y2 = int(input("Enter y2:"))
bresenham(x1, y1, x2, y2)
```

24/2/26

result:- Thus the python program to Implement Bresenham's Line Drawing algorithm for drawing a line with $|m| < 1$ was sucessfully executed and verified.

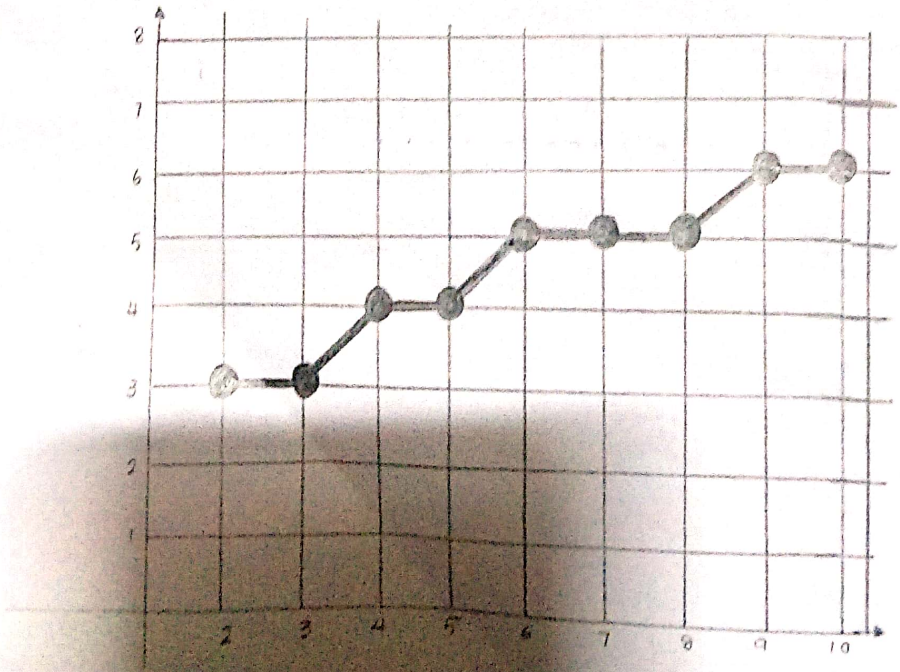
Scanned with OKEN Scanner

Sample Input:-

```
Enter x1: 2
Enter x2: 3
Enter y1: 10
Enter y2: 6
```

Sample Output:-

```
points on the line:
(2,3)
(3,3)
(4,4)
(5,4)
(6,4)
(7,5)
(8,5)
(9,6)
(10,6)
```



Scanned with OKEN Scanner

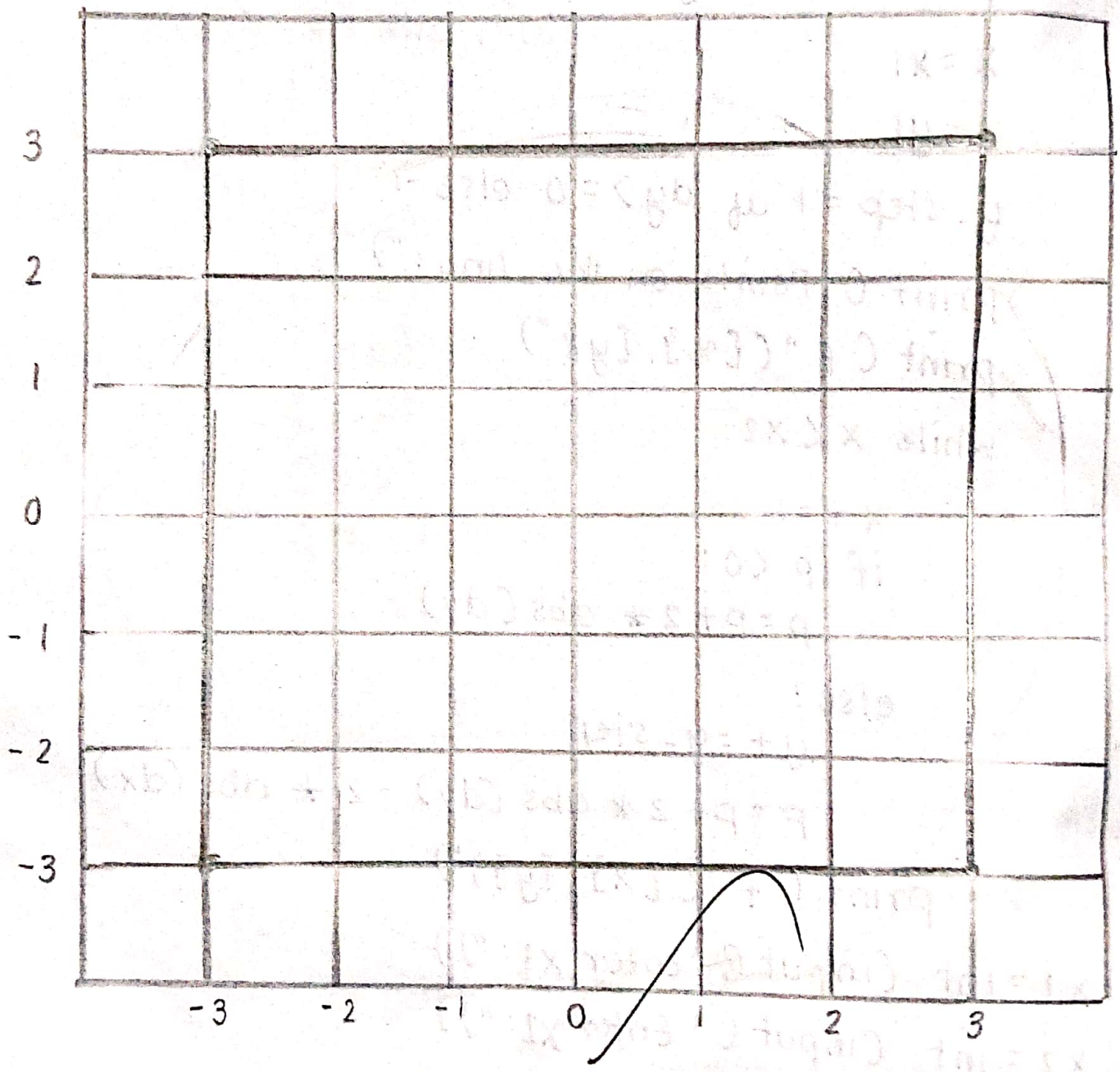
Sample Input:-

Enter center x-coordinate: 0

Enter center y-coordinate: 0

Enter radius: 3

Sample Output:-



Scanned with OKEN Scanner

24/7/26 Experiment-2 Bresenham's Square Drawing algorithm

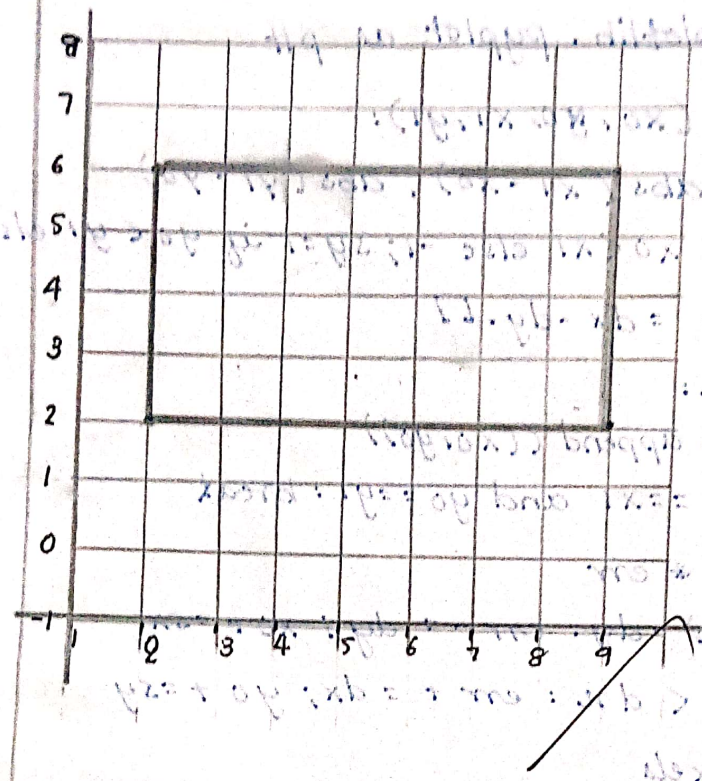
aim:- Theory is to Implement a square using the Bresenham's algorithm

program:-

```
import matplotlib.pyplot as plt
def bresenham(x0, y0, x1, y1):
    dx, dy = abs(x1 - x0), abs(y1 - y0)
    sx = 1 if x0 < x1 else -1; sy = 1 if y0 < y1 else -1
    err, pixels = dx - dy, []
    while True:
        pixels.append((x0, y0))
        if x0 == x1 and y0 == y1: break
        e2 = 2 * err
        if e2 > -dy: err -= dy; x0 += sx
        if e2 < dx: err += dx; y0 += sy
    return pixels
def draw_square(x, y, s):
    p = bresenham(x, y, x+s, y)
    p.extend(bresenham(x+s, y, x+s, y+s))
    p.extend(bresenham(x+s, y+s, x, y+s))
    p.extend(bresenham(x, y+s, x, y))
    return p
x_s, y_s, size = 10, 10, 50
square_px = draw_square(x_s, y_s, size)
plt.figure(figsize=(8,8))
plt.plot(*zip(*square_px), color='green')
plt.xlim(0,100); plt.ylim(0,100)
plt.gca().set_aspect('equal', adjustable='box')
plt.title(f'Bresenham Square ({x_s}, {y_s})
                    size {size} - smaller code')
                    alpha=0.7); plt.show
```

Scanned with OKEN Scanner

Output :-



1/8/22

Exercise - Draw a simple Rectangle using Bresenham algorithm.

aim:- Drawing a rectangle using Bresenhams algorithm with the top left point as (2,2) and the height to be 5 cm and width is 8 cm.

code:-

```
import matplotlib.pyplot as plt
x_cord = [p[0] for p in rectangle_points]
y_coord = [p[1] for p in rectangle_points]
fig = plt.figure (figsize=(8,6))
plt.scatter (x_cord, y_coord, color='blue', s=10,
                                label='Rectangle')

plt.plot ([x1, x2], [y1, y1], color='red', line width=2)
plt.plot ([x2, x2], [y1, y2], color='red', line width=2)
plt.plot ([x2, x1], [y2, y2], color='red', line width=2)
plt.plot ([x1, x1], [y2, y1], color='red', line width=2)
plt.xlabel ('x')
plt.ylabel ('y')
plt.grid (True)
plt.gca().set_aspect ('equal', adjustable='box')
plt.xticks (np.arange (min (x_cord)-1, max (x_cord)+2, 1))
plt.yticks (np.arange (min (y_coord)-1, max (y_coord)+2, 1))
plt.legend()
plt.show()
```

result:-

Thus a Rectangle was drawn using Bresenham's algorithm and was verified.

Scanned with OKEN Scanner

Scanned with OKEN Scanner

Exercise - Drawing the Olympic logo using mid-point algorithm

aim:- To implement the Olympic logo using the mid-point circle algorithm.

program:-

```
import matplotlib.patches as patches
fig = plt.figure (figsize = (10,6))
ax = fig.add-subplot (111)
ring-zorder = {'blue': 3, 'yellow': 4, 'black': 5,
               'green': 2, 'red': 4}

for config in ring-configs:
    cx, cy = config ['center']
    r = config ['radius']
    color = config ['color']
    zorder = ring-zorders.get (color, 0)
    white-halo = patches.Circle ((cx, cy), r + 0.5,
                 facecolor = 'none', edgecolor = color,
                 linewidth = 15, zorder = zorder)

ax.add-patch (circle)
ax.set-title ('Olympic logo')
ax.set-xlabel ('x-coordinate')
ax.set-ylabel ('y-coordinate')
ax.set-aspect ('equal', adjustable = 'box')
ax.grid (True, linestyle = '--', alpha = 0.7)
min-x = min ((c['center'][0] - c['radius']) for c in
                                    ring-configs)
max-x = max(c['center'][0] + c['radius'] for c in
                                    ring-configs)
min-y = min ((c['center'][1] + c['radius'] for c in
                                    ring-configs)
max-y = min ((c['center'][1] + c['radius'] for c in
                                    ring-configs)

ax.set-xlim (min-x - padding, max-x + padding)
ax.set-ylim (min-y - padding, max-y + padding)
plt.savefig ('/content/olympic.png')
plt.show ()
```

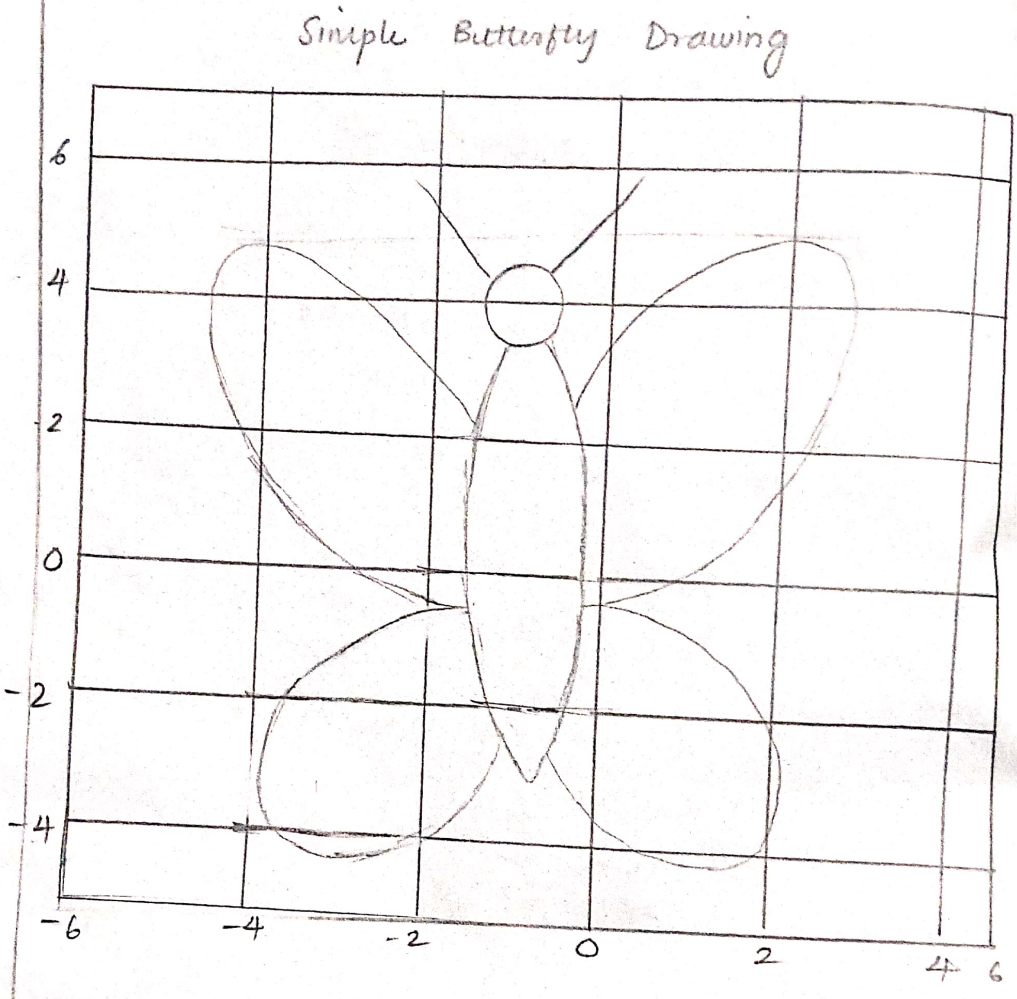
Scanned with OKEN Scanner

Result:- Thus the olympic logo was created using mid-point algorithm and was verified successfully.



Scanned with OKEN Scanner

Output :-



Scanned with OKEN Scanner

Exercise Drawing Butterfly using mid-point algorithm

aim:- To draw a Butterfly usind mid-point algorithm with proper proposiions and color.

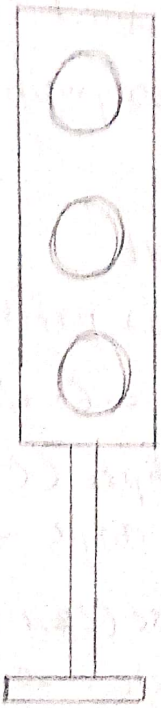
program:-

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches
fig, ax = plt.subplots(figsize=(6,6))
ax.add_patch(patches.ellipse((-2,1.5),4,8,
                             angle=30, color='purple'))
ax.add_patch(patches.ellipse((2,1.5),4,8,
                             angle=-30, color='purple'))
ax.add_patch(patches.ellipse((-2,-2),3.5,5,
                             angle=-20, color='lightpink'))
ax.add_patch(patches.ellipse((2,-2),3.5,5, angle=20,
                             color='lightpink'))
ax.add_patch(patches.ellipse((0,0),1.4,8, angle=0,
                             color='black'))
ax.add_patch(patches.ellipse((0,4.5),1.5,1.5,
                             angle=0, color='black'))
plt.plot([-0.4,-1.5],[5.25,6.5],'k',
                             linewidth=2)
plt.plot([0.4,1.5],[5.25,6.5],'k',
                             linewidth=2)
ax.set_title('simple Butterfly Drawing')
plt.axis('equal')
plt.grid(True)
plt.show()
```

result:- Thus the Butterfly was drawn successfully using mid-point algorithm and is verified.

Scanned with OKEN Scanner

Output :-



Scanned with OKEN Scanner

Exercise Drawing Traffic Signal

aim:- To Draw the Traffic Signal using Mid-point algorithm with proper proposition and color

program:-

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches
fig,ax = plt.subplots(figsize=(4,12))
body_width, body_height = 3.5, 10
ax.add_patch(patches.Rectangle((-body_width/2,0),
        body_width,body_height,facecolor='#333333',
    edgecolor='black',linewidth=2))

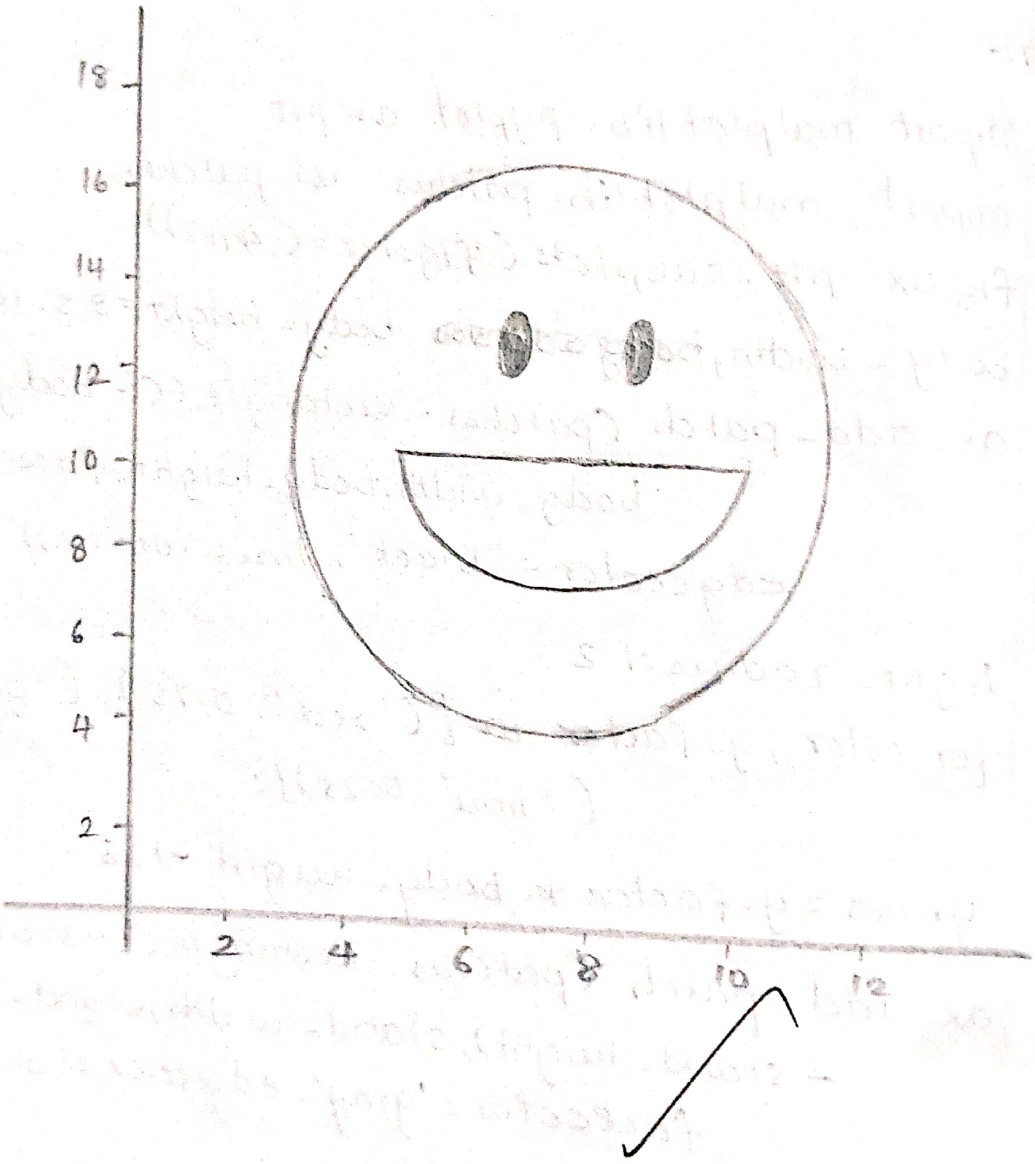
light_radius=1.2
for color,y_factor in [('red',0.75),('gold',0.5),
        ('lime',0.25)]:
    y_pos = y_factor * body_height =1,8
ax.add_patch(patches.Rectangle((-stand_width/2),
    -stand_height),stand_width,stand_height,
        facecolor='gray',edgecolor='black',linewidth=1)
base_width,base_height = 5,1
ax.add_patch(patches.rectangle((-base_width/2,
    -stand_height-base_height),base_width,base_height
        facecolor='dimgray',edgecolor='black',
            linewidth=2))
ax.set_aspect('equal'),adjustable='box')
ax.set_xlim(-6,6)
ax.set_ylim(-10-stand_height,body_height+light_radius
                                        +1)
ax.axis('off')
plt.show
```

result:-

Thus the Traffic Signal was implemented successfully with the mid-point algorithm

Scanned with OKEN Scanner

Output:-



Scanned with OKEN Scanner

Exercise — Smiley face using Mid-Point Algorithm.

aim:- To draw a simple smiley face using mid-point algorithm with proper proportion and color

program:-

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches
fig, ax = plt.subplots(figsize=(6,6))
head = patches.Circle((0.5,0.5), 0.4, facecolor =
        'yellow', edgecolor='black', linewidth=2)
ax.add_patch(head)
img = np.full((500, 600, 3), 255, dtype=np.uint8)
black = [0,0,0]
def midpoint_circle(xc, yc, r):
    point = []
    x = 0
    y = r
    p = 1 - r
    while x <= y:
        points.extend([[(xc + x, yc + y),
            (xc - x, yc + y), (xc + x, yc - y),
            (xc - x, yc - y), (xc + y, yc + x),
            (xc - y, yc + x), (xc + y,
                yc - x),
            (xc - y, yc - x)]])

        if p < 0:
            p += 2 * x + 3
        else:
            p += 2 * (x - y) + 5
            y -= 1
        x += 1
    return points
```

Scanned with OKEN Scanner

Exercise

```
def filled_circle(xc, yc, r, color):
    y = np.arange(yc-r, yc+r, 0.01)
    for yy in y:
        half = np.sqrt(r*r - (yy-yc)**2)
        plt.plot([xc-half, xc+half],
                 [yy, yy], color=color)
    t = np.linspace(0, 2*np.pi, 300)
    x = xc + r*np.cos(t)
    y = yc + r*np.sin(t)
    plt.plot(x, y, color="black", linewidth=5)
filled_circle(0,0,5,"yellow")
filled_circle(-1.8, 1.5, 0.5, "black")
filled_circle(1.8, 1.5, 0.5, "black")
t = np.linspace(np.pi+0.2, 2*np.pi-0.2,
                150)
x = 2.5*np.cos(t)
y = 1.5*np.sin(t) - 0.5
plt.show()
```

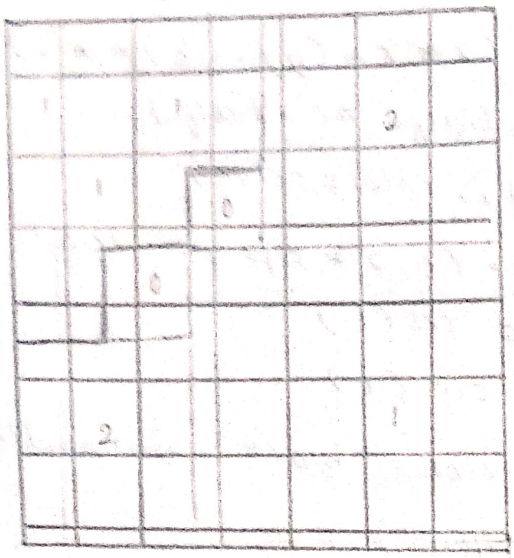


result:- Thus a smiley face was drawn using mid point algorithm and was displayed succesfully.

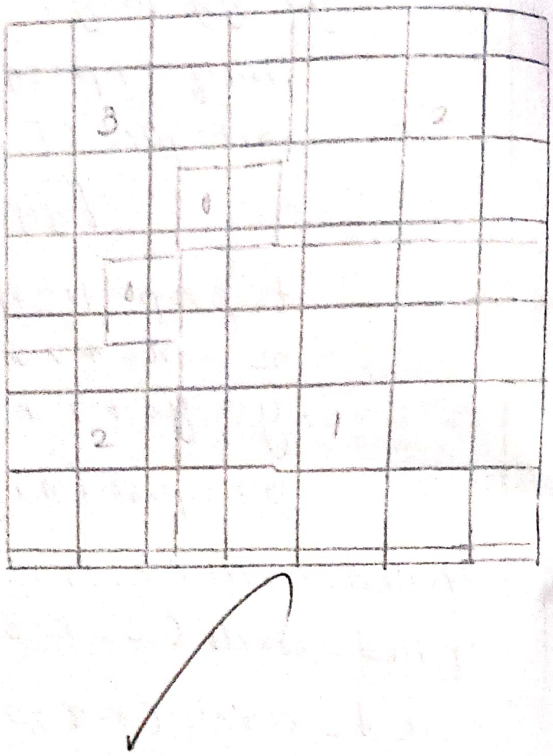
Scanned with OKEN Scanner

Output:-

Before



After 1 → 8



Experiment - 2 Flood Fill Algorithm

aim:- to Implement the flood fill algorithm to fill a connected region with a new color

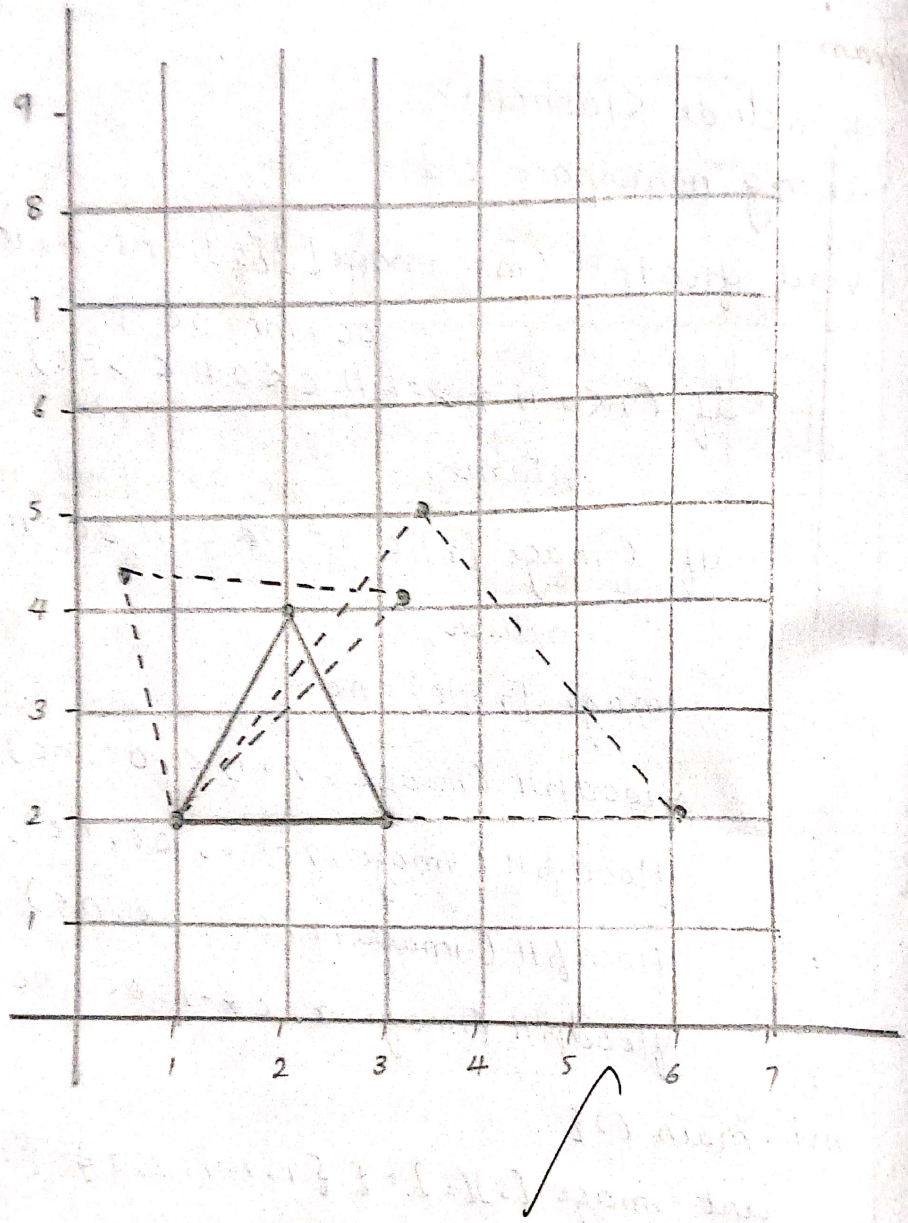
program:-

```
#include <iostream>
using namespace std;
void floodfill (int image[][5], int r, int c, int
                oc, int nc
    if (r<0 || r>=5 || c<0 || c>=5)
        return;
    if (image[r][c] != oc || oc == nc)
        return;
    image[r][c] = nc;
    floodfill (image, r+1, c, oc, nc);
    floodfill (image, r-1, c, oc, nc);
    floodfill (image, r, c+1, oc, nc);
    floodfill (image, r, c-1, oc, nc); }
int main (){
    int image[5][5] = { {1,1,1,2,2}, {1,1,0,2,2},
                        {1,0,1,1,1}, {2,2,1,1,1},
                        {2,2,1,1,1} };
    floodfill (image, 0, 0, 1, 3);
    for (int i=0; i<5; i++){
        for (int j=0; j<5; j++)
            cout << image[i][j] << " ";
        cout << endl; }
    return 0; }
```

result:- Thus the Flood Fill Algorithm was Implemented and the Colored Region was filled with the Specified Color.

Scanned with OKEN Scanner

Output:-



Scanned with OKEN Scanner

Experiment : 8A

Implement Transformations like Translation, Scaling and Rotations to 2D shapes using Transformation Matrices.

aim:- To Implement Translation, Scaling & Rotation to 2D shapes using Transformation Matrices

program:-

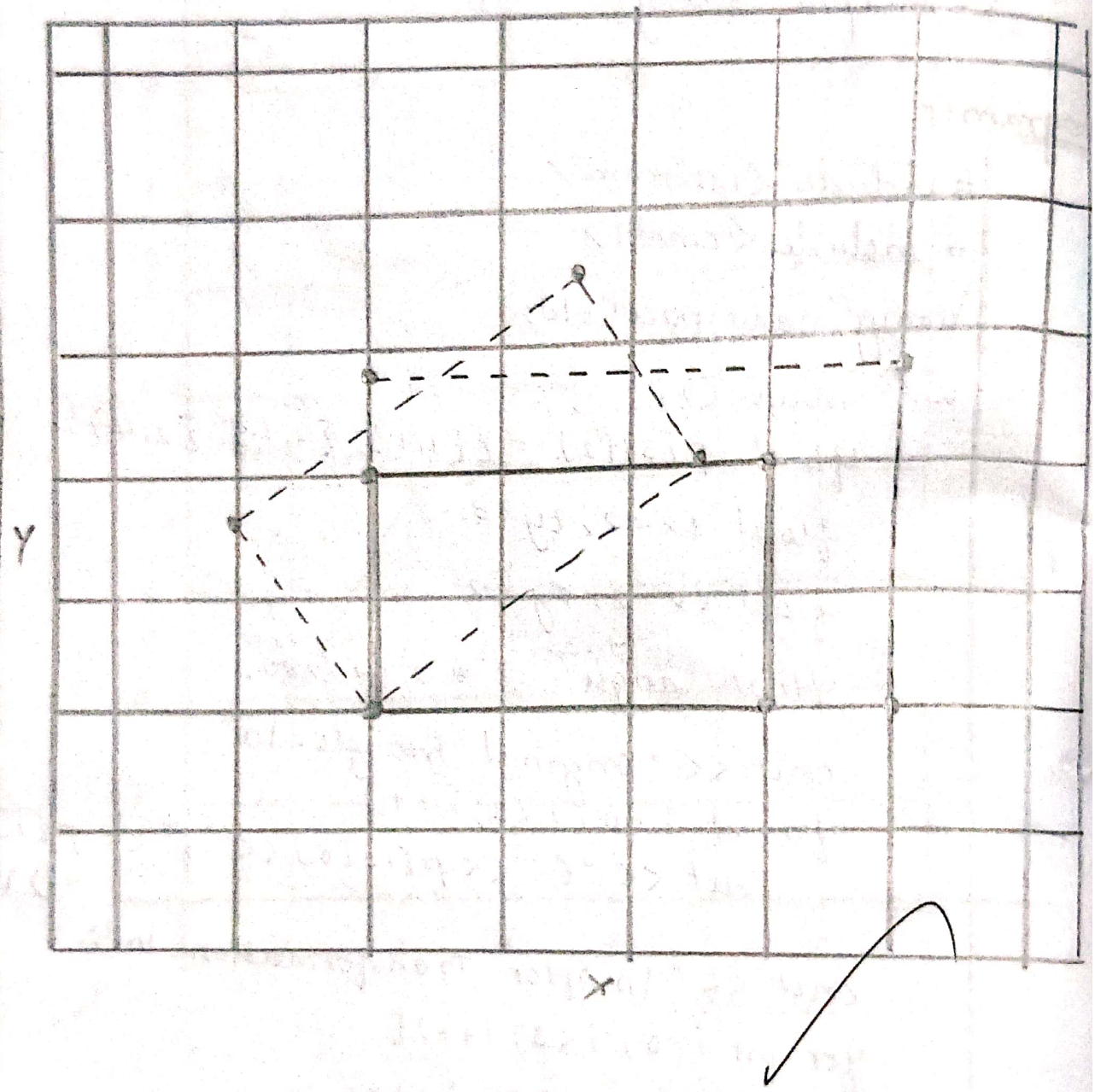
```
#include <iostream>
#include <cmath>
using namespace std;
int main(){
    float p[3][2] = {{1,1},{4,1},{2,4}};
    float tx=2, ty=3;
    float sx=2, sy=2;
    float angle = 30*3.14/180;
    cout<<"original triangle:\n";
    for(int i=0;i<3;i++)
        cout<<"("<<p[i][0]<<","<<p[i][1]<<")\n";

    cout<<"\nAfter Transformation:\n";
    for(int i=0;i<3;i++){
        float x = p[i][0]+tx;
        float y = p[i][1]+ty;
        x*=sx; y*=sy;
        float X = x*cos(angle)-y*sin(angle);
        float Y = x*sin(angle)+y*cos(angle);
        cout<<"("<<X<<","<<Y<<")\n";}
    return 0;
}
```

result:- Thus the Triangle was successfully transformed using the given 2D transformations.

Scanned with OKEN Scanner

output :-



Scanned with OKEN Scanner

Experiment 5b.

Transformation of a Rectangle.

aim:- To perform transformation to a 2D rectangle using transformation matrices

program:-

```
import numpy as np
import matplotlib.pyplot as plt
        [1,4,4,1,1], [1,1,3,3,1], [1,1,1,1,1]

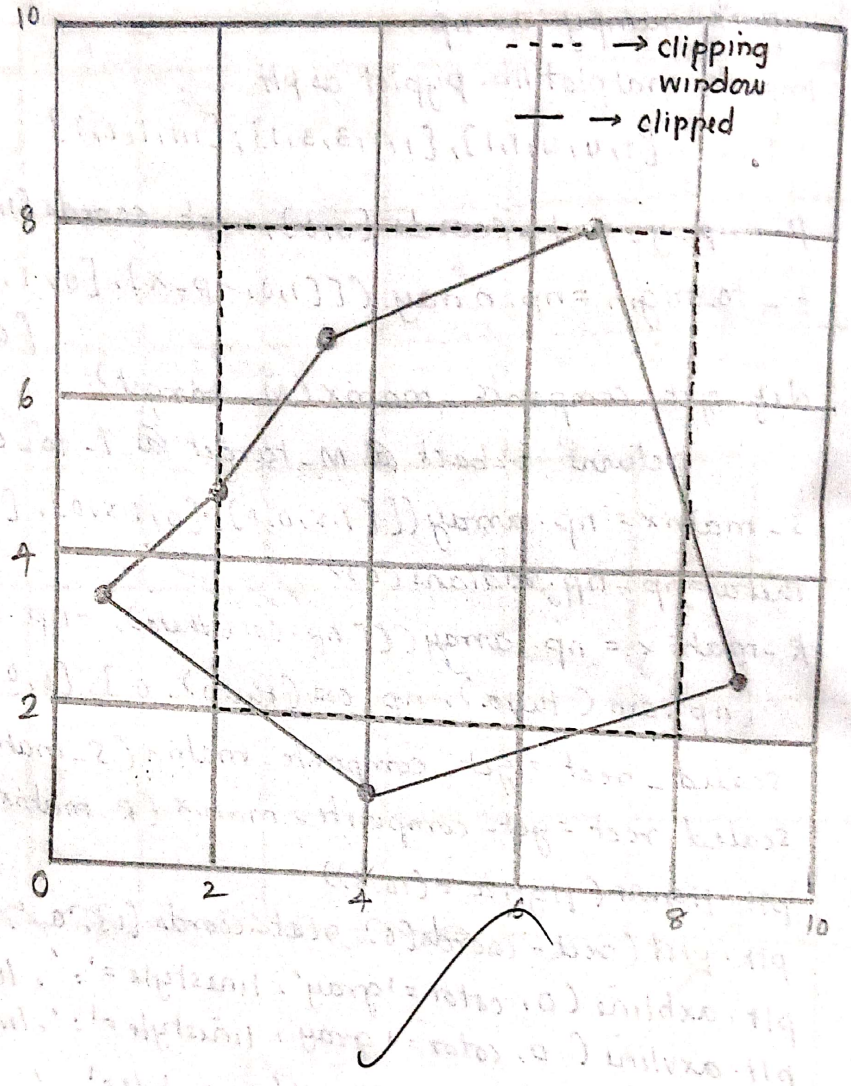
p-x, p-y = rect-coords[0,0], rect-coords[1,0]
t-to_origin = np.array([[1,0,-p-x], [0,1,p.y],
                                    [0,0,1]])
def get-composite-matrix(M-target):
        return t-back @ M-target @ T-to-origin
S-matrix = np.array([[1.5,0,0], [0,1.5,0], [0,0,1]])
theta = np. np.radians(45)
R-matrix = np.array([[np.cos(theta), -np.sin(theta), 0],
  [np.sin(theta), np.cos(theta), 0], [0,0,1]])
scaled_rect = get_composite-matrix(S_matrix)@rect-
scaled-rect = get-composite-matrix(R_matrix)@rect-coords
                                            coords
plt.figure(figsize=(10,8))
plt.plot(rect-coords[0], rect-coords[1], 'o-', label=
plt.axhline(0, color='gray', linestyle=':', linewidth=0.8)
plt.axvline(0, color='gray', linestyle=':', linewidth=0.8)
plt.grid(True, which='both', linestyle='--', alpha=0.5)
plt.legend()
plt.xlabel('x')
plt.ylabel('y')
plt.axis('equal')
plt.show()
```

Result:-

Thus the Rectangle was Transformed successfully using the given 2D transformation.

Scanned with OKEN Scanner

Output:-



Experiment 4 Polygon Clipping Algorithm

aim:- To develop a program that clips polygons against a specified clipping window using polygon clipping algorithm.

result:-

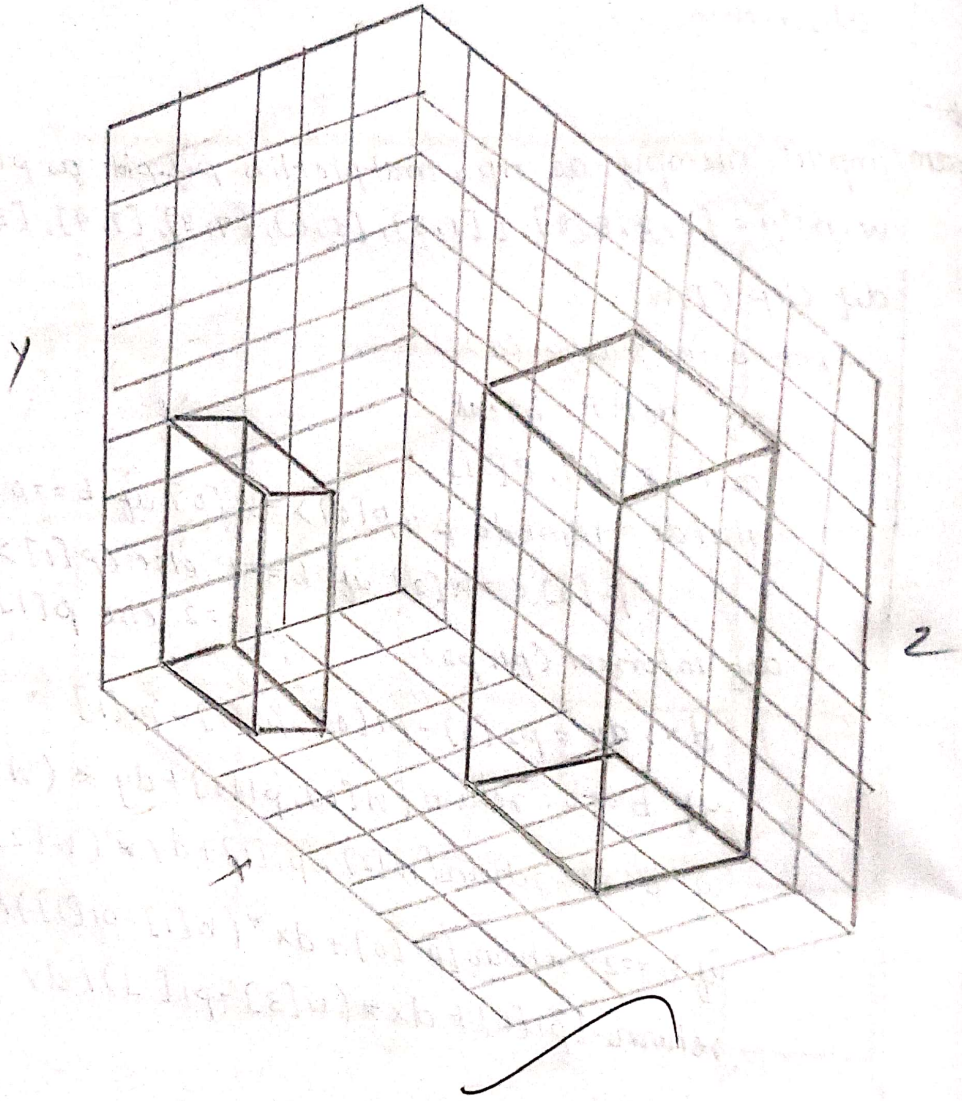
Program:

```
import numpy as np, matplotlib.pyplot as plt
w,poly = [2,2,8,8],[[1,3],[5,0],[9,5],[7,9],[3,7]]
def clip(P,W):
    for b in range(4):
        if not P: break
        out,s = [],P[-1]
        inside = lambda p: p[0]>=W[0] if b==0 else
                 (p[0]<=W[2] if b==1 else (p[1]>W[1] if
                 b==2 else p[1]<=W[3]))
        def intersect(p1,p2):
            dx,dy = p2[0]-p1[0],p2[1]-p1[1]
            if b==0: return [W[0],p1[1]+dy*(W[0]-p1[0])/
                            dx]
            if b==1: return [W[2],p1[1]+dy*(W[2]-p1[0])/dx]
            if b==2: return [p1[0]+dx*(W[1]-p1[1])/dy,W[1]]
            return [p1[0]+dx*(W[3]-p1[1])/dy,W[3]]
        for p in P:
            if inside(p):
                if not inside(s): out.append(intersect
                                  (s,p))
                out.append(p)
            elif inside(s): out.append(intersect(s,p))
            s=p
        P=out
    return P
c=clip(poly,w)
plt.xlim(0,10);plt.ylim(0,10);plt.grid(True);plt.legend();
                                  plt.show()
```

result:- Thus a program to clip a polygon against a specified clipping window was done & verified

Scanned with OKEN Scanner

Output:-



Scanned with OKEN Scanner

Exp 5: 3D Transformation on Basic objects - Cube

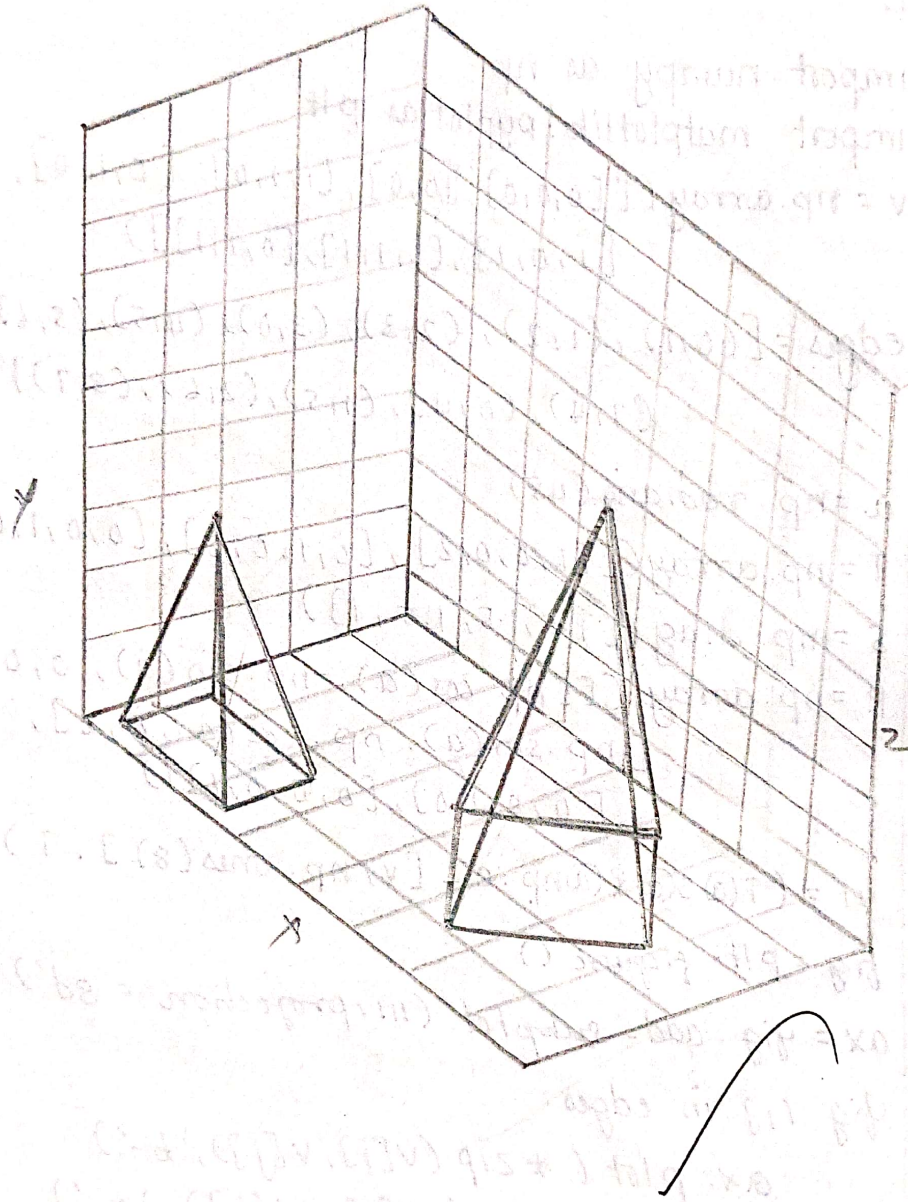
aim:- To write a program that allows a user to perform 3D transformations on a basic 3D object cube

program:-

```
import numpy as np
import matplotlib.pyplot as plt
v = np.array([[0,0,0],[0,0,0],[1,1,0],[0,1,0],[0,0,1],
              [1,0,1],[1,1,1],[0,1,1]])
edges = [(0,1),(1,2),(2,3),(3,0),(4,5),(5,6),(6,7),
         (7,4),(0,4),(1,5),(2,6),(3,7)]
a = np.radians(45)
T = np.array([[1,0,0,2],[0,1,0,2],[0,0,1,0],[0,0,0,1]])
S = np.diag([1.5,1.5,1.5,1])
R = np.array([[np.cos(a),-np.sin(a),0,0],
              [np.sin(a),np.cos(a),0,0],
              [0,0,1,0],[0,0,0,1]])
V1 = (T@S@R@np.c_[v,np.ones(8)].T).T[:,:3]
fig = plt.figure()
ax = fig.add_subplot(111,projection='3d')
for i,j in edges:
    ax.plot(*zip(v[i],v[j]),'b-')
    ax.plot(*zip(V1[i],V1[j]),'r-')
ax.set_title("3D cube transformation")
plt.show()
```

result:- Thus the Program was Executed successfully & the Output was verified.

Scanned with OKEN Scanner



Scanned with OKEN Scanner

Exp 5 b. 3D Transformation of Pyramid

aim:- To perform 3D transformation to a pyramid using a program.

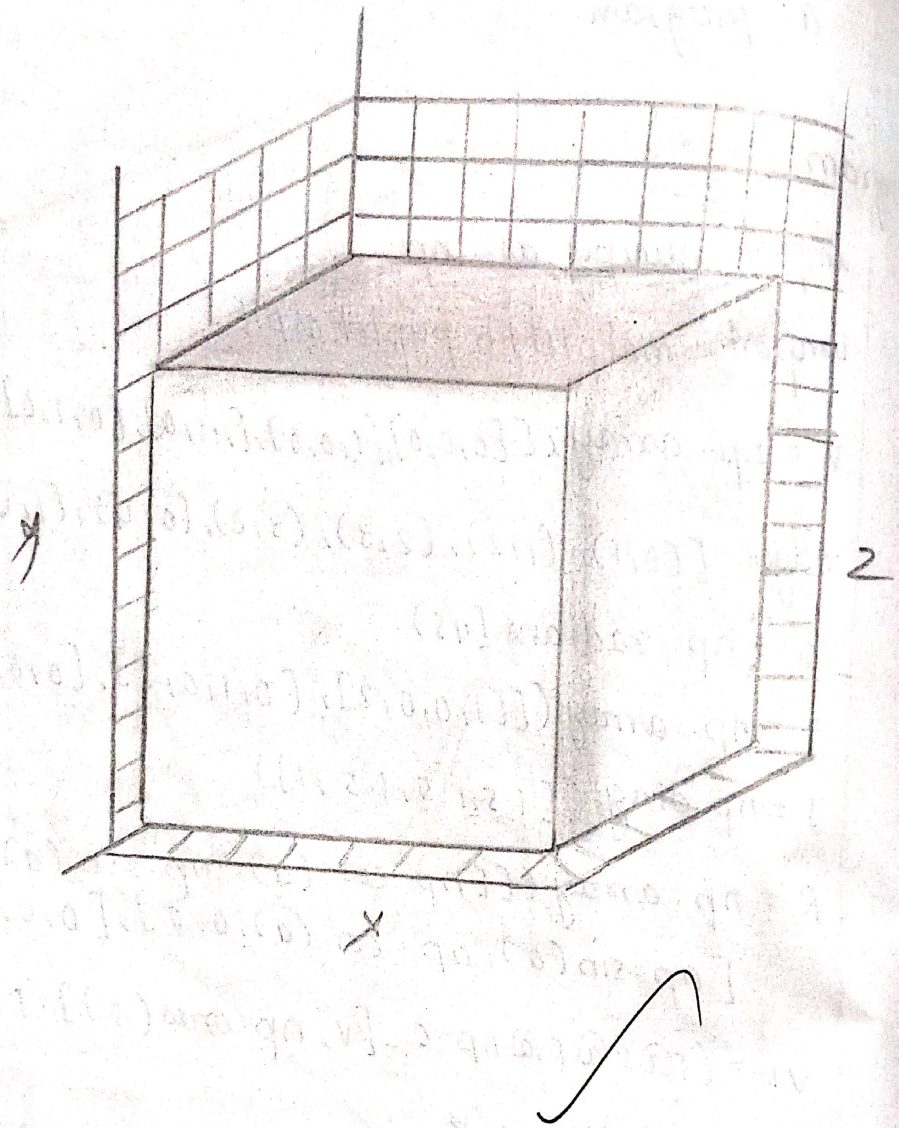
program:-

```
import numpy as np
import matplotlib.pyplot as plt
V = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0],[0.5,0.5,1.5]])
edges = [(0,1),(1,2),(2,3),(3,0),(0,4),(1,4),(2,4),(3,4)]
a = np.radians(45)
T = np.array([[1,0,0,2],[0,1,0,2],[0,0,1,0],[0,0,0,1]])
S = np.diag([1.5,1.5,1.5,1])
R = np.array([[np.cos(a),-np.sin(a),0,0],
    [np.sin(a),np.cos(a),0,0],[0,0,1,0],[0,0,0,1]])
V1 = (T@S@R@np.c_[V,np.ones(5)].T).T[:,:3]
fig = plt.figure()
ax = fig.add_subplot(111,projection='3d')
for i,j in edges:
    ax.plot(*zip(V[i],V[j]),'b-')
    ax.plot(*zip(V1[i],V1[j]),'r-')
plt.show()
```

result:- Thus the program was written and the output was verified successfully

Scanned with OKEN Scanner

Output:-



Scanned with OKEN Scanner

Experiment - 6 Create & Render 3D objects using Polygons & applying colors & shading.

aim:- To create and Render a 3D object using polygons and to apply colors and shading.

program:-

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import poly3d
                                       collection

import numpy as np
v = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0],[0,0,1],
              [1,0,1],[1,1,1],[0,1,1]])

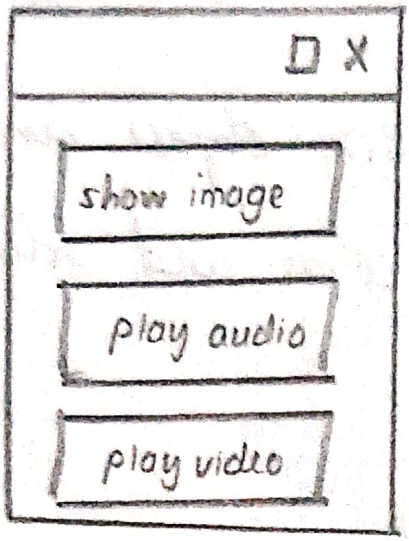
faces = [[v[j] for j in [0,1,2,3]],
    [v[j] for j in [4,5,6,7]],
    [v[j] for j in [0,1,5,4]],
    [v[j] for j in [2,3,7,6]],
    [v[j] for j in [1,2,6,5]],
    [v[j] for j in [4,7,3,0]]]
```

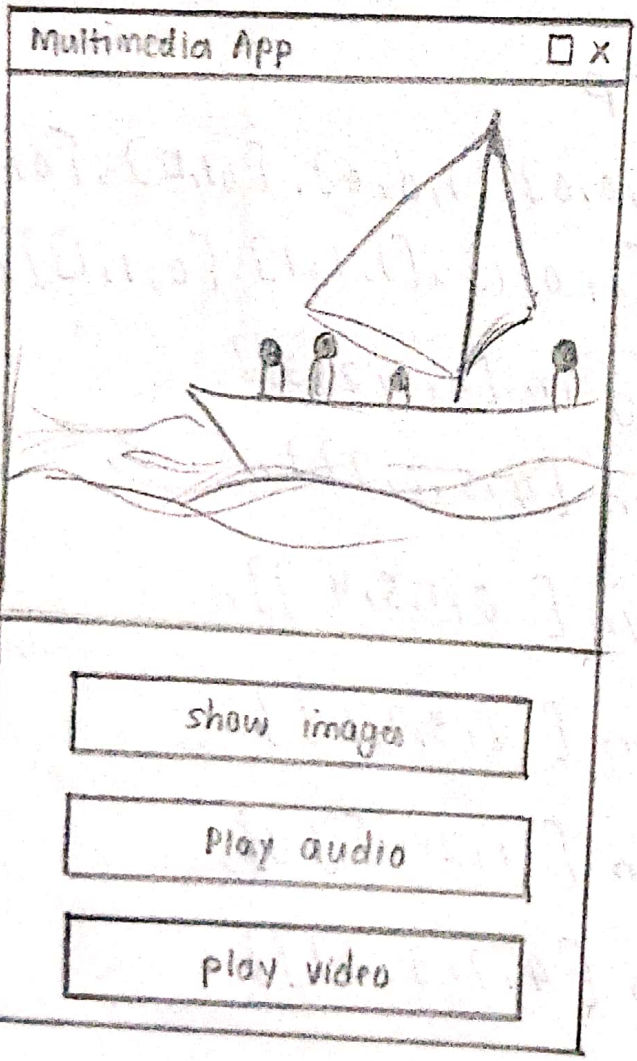
Result:-

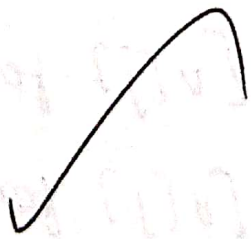
Thus the 3D polygon was created, Rendered and colors and shading were applied to it successfully.

Scanned with OKEN Scanner

Output :-







Scanned with OKEN Scanner

Experiment 7 Create a Multimedia Application

aim:- to develop a simple multimedia application that integrates • Images • Audio • Video into a unified user UI using python & suitable libraries

program:-

```
import tkinter as tk
from PIL import Image, ImageTk
from playsound import playsound
import cv2, threading.

root = tk.Tk()
root.title("Multimedia App")
def image():
    img = ImageTk.PhotoImage(Image.open(
                "sample-image.jpg"))
    label.image = img
    label.config(image = img)
def audio():
    threadig.Thread(target=playsound, args("sample-
                audio.mp3",)).start()
def video():
    cap = cv2.VideoCapture("sample_video.mpu")
    while cap.isopened():
        ret, frame = cap.read()
        if not ret: break
        cv.imshow("Video", frame)
        if (cv2.waitkey(25) == ord('q'): break
        cap.release()
    cv2.destroyAllWindow()
label = tk.label(root)
label.pack()
```

Scanned with OKEN Scanner

```
tk. Button(root, text = 'Image", command = image)
                                        pack()
tk. Button(root, text = 'Audio', command = audio). pack()
tk. Button(root, text = 'Video',
        command = lambda : threading. Thread
            (Target = video). start()). pack()

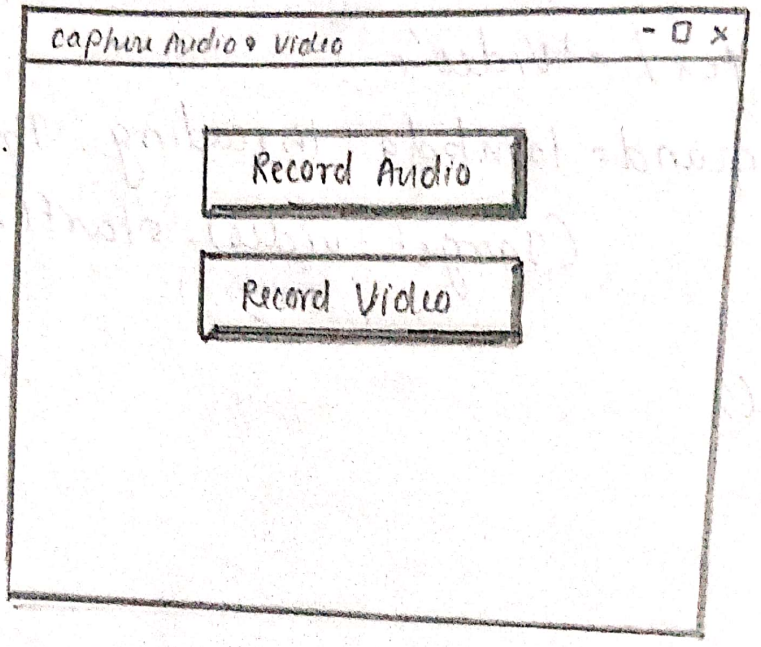
root. mainloop()
```

result:-

A Multimedia application was created and it successfully integrates Images, Videos and Audios.

Scanned with OKEN Scanner

Output:-




```
Recording Audio...
Audio recording saved.
Press 'q' to stop video.
Recording audio...
Audio recording saved.
```

Scanned with OKEN Scanner

Experiment 8: Capture video/Audio and display on Multimedia Interface.

aim:- To write a program to capture Video/Audio and display it on a multimedia Interface

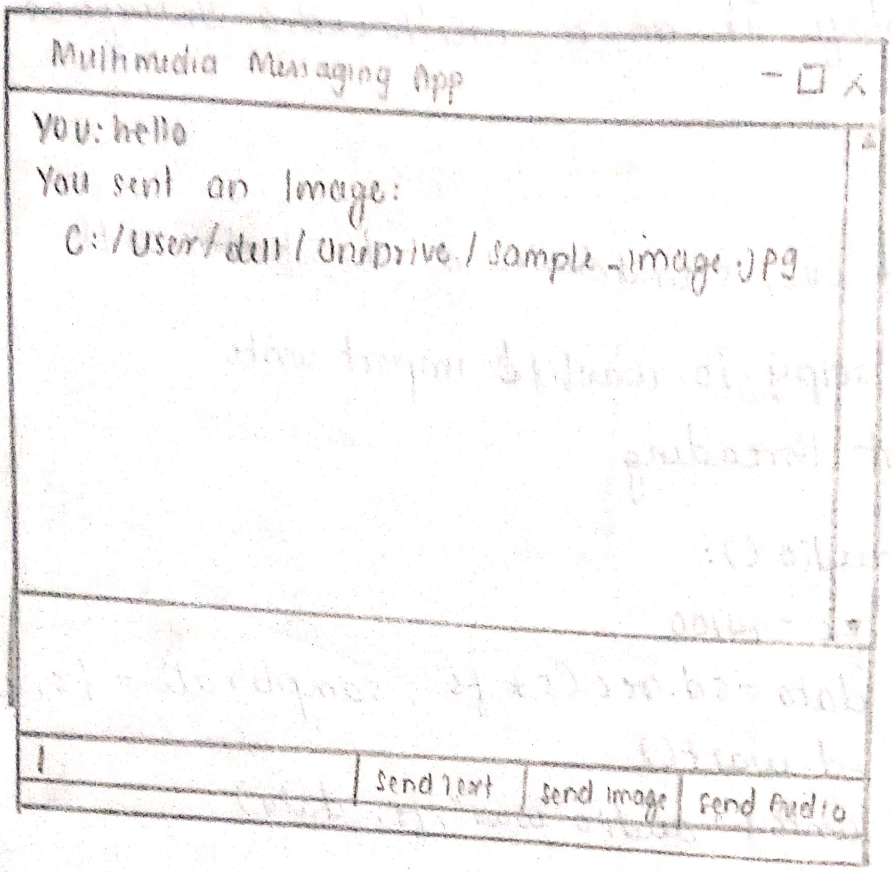
program:-

```
import cv2, sounddevice as sd, tkinter as tk
from scipy.io.wavfile import write
import threading
def audio():
    fs = 44100
    data = sd.rec(5 * fs, samplerate = fs, channels=2)
    sd.wait()
    write("audio.wav", fs, data)
def video():
    cap = cv2.videocapture(0)
    while True:
        ret, frame = cap.read()
        if not ret: break
        cv.imshow("Webcam", frame)
        if cv.waitkey(1) == ord('q'): break
cap.release()
cv2.destroyallWindow()
root = tk.Tk()
tk.Button(root, text='Record Audio', command= lambda:
                threading.Thread(target=audio).start().
                pack()
tk.Button(root, text='Record Video', command= 'lambda: Threading
                .Thread(target=video).start().pack()
root.mainloop()
```

result:- Thus the Program was created successfully and the output was verified.

Scanned with OKEN Scanner

Output:-


Multimedia Messaging App
You: hello
You sent an Image:
C:/Users/dell/OneDrive/Sample_image.JPG
Send Text
Send Image
Send Audio

Experiment 9: Create an Application to Send & Receive Multimedia Messages (Text, Images & Video)

aim:- to Develop a basic chat-like application that allows sending text, image and audio using Tkinter file selection and simulated messaging UI

program:-

```
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageTk
from playsound import playsound
import threading

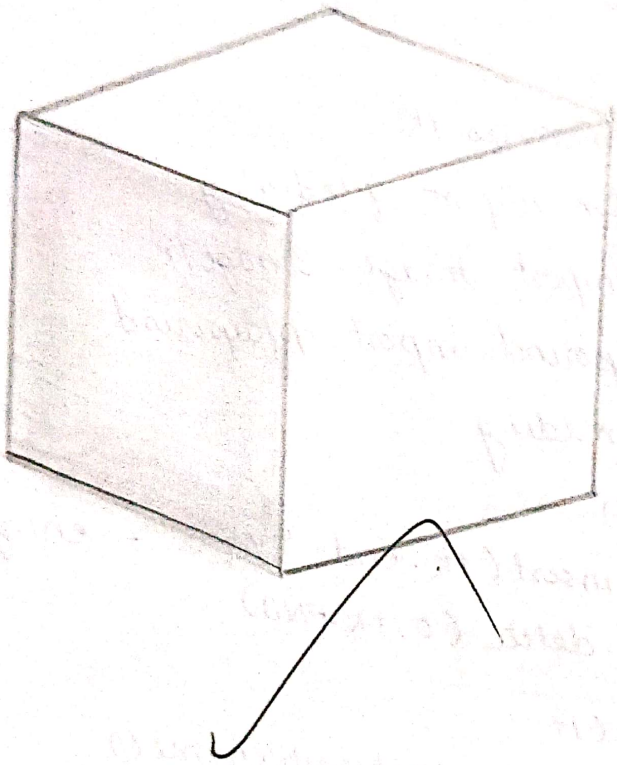
def text():
    chat.insert(tk.END, "You: " + entry.get() + "\n")
    entry.delete(0, tk.END)

def image():
    p = filedialog.askopenfilename()
    if p:
        img = Image.open(p)
        img.thumbnail((100,100))
        pic = ImageTk.PhotoImage(img)
        label.config(image=pic)
        label.image = pic

def audio():
    p = filedialog.askopenfilename()
    if p:
        threading.Thread(target=playsound, args=(p,),
                         daemon=True).start()
tk.Button(w, text="Text", command=text).pack()
tk.Button(w, text="Image", command=image).pack()
tk.Button(w, text="Video", command=video).pack()
w.mainloop()
```

result:- Thus a multimedia interface was created and run successfully and was verified

Output :-



Scanned with OKEN Scanner

Experiment 10: Create a 3D model using Blender

aim:- to create a simple 3D model using Blender and apply Basic shading and Textures

procedure:-

1. launch Blender
2. Model Creation:
   - use Add → Mesh to Insert a Basic shape like cube
   - combine them to form a house
   - use edit Mode to extrude / shape.
3. shading:
   - switch to shading workspace
   - add a new material for each object
   - Adjust Base color, Roughness & specular.
4. Texture Mapping:-
   - in the shader Editor, add an Image texture node
   - load an External image.
   - Connect it to the Base color of the principled BSDF shader
   - use UV Editing to align
5. Lighting and camera.
   - add light sources (sun/point light)
   - Position the camera to frame the model
6. Render Settings:
   - choose Cycles or Eevee rendering engine
   - Adjust resolution & Sampling
   - click Render → Render Image to produce output.

Result:- Thus the Process of shading was done successfully using Blender.

Scanned with OKEN Scanner